VesperaRoom — Normalización del modelo de datos

Oct 27, 2026 · @Felipe

1. Introducción y conceptos base

El modelo de VesperaRoom queda en **tercera forma normal (3FN) con 19 tablas**. Todas cumplen además Boyce-Codd, salvo `habitacion`, que se deja en 3FN a propósito. 4FN y 5FN se cumplen sin descomponer nada más. El punto de partida es el diagrama ER en notación Chen (`diagrama_ER_motel.drawio`): se le aplican cinco ajustes y luego se recorre la normalización paso a paso.

Conceptos que se usan en todo el documento

Concepto	Qué significa	Ejemplo en VesperaRoom
Dependencia funcional (DF) X → Y	Si dos filas tienen el mismo X, tienen obligatoriamente el mismo Y	`correo → nombre`
Superclave	Conjunto de atributos que identifica una fila de forma única	`{id_usuario, correo}`
Clave candidata	Superclave mínima: si se le quita un atributo, deja de identificar	`id_usuario`, `correo` y `telefono` en `usuario`
Clave primaria (PK)	La clave candidata elegida para identificar la tabla	`id_usuario`
Atributo primo	Atributo que forma parte de *alguna* clave candidata	`id_motel` en la clave `{id_motel, numero}` de `habitacion`
Dependencia parcial	Un atributo no primo depende de solo una *parte* de una clave compuesta. Viola 2FN	`(id_reserva, id_servicio) → nombre_servicio`
Dependencia transitiva	A → B → C, donde B no es clave. Viola 3FN	`id_motel → ciudad → departamento`

Por qué normalizar: las tres anomalías

Cuando un mismo hecho se guarda en varias filas aparecen tres anomalías:

- **De inserción:** no se puede crear un servicio adicional si nadie lo ha comprado todavía.
- **De actualización:** cambiar el nombre de un servicio obliga a editar cientos de filas. Si se olvida alguna, la base queda contradictoria.
- **De borrado:** al borrar la única reserva de una habitación se pierden los datos de la habitación.

Normalizar es descomponer las tablas hasta que **cada hecho se guarde una sola vez**. Las descomposiciones deben ser **sin pérdida**: al unir las tablas con JOIN se recupera exactamente la información original.

2. Ajustes al modelo ER

Antes de normalizar se corrigen cinco puntos del diagrama original. Ninguno afecta a la tabla `usuario`, que ya está implementada.

#	Problema en el diagrama	Ajuste
1	RESERVA no sabía qué habitación concreta se reservó; solo llegaba al tipo a través de la tarifa. Sin esto no se puede controlar que no haya dos reservas en la misma habitación a la misma hora.	Nueva relación **HABITACION 1:N RESERVA**, que se traduce en `reserva.id_habitacion`.
2	PERFIL_PROPIETARIO y POLITICA_RESERVA no tenían PK.	Son relaciones 1:1, así que su PK es la FK de su dueño: `perfil_propietario.id_usuario` y `politica_reserva.id_motel`.
3	FOTO dependía de MOTEL y de HABITACION a la vez.	Dos FK que admiten NULL y un `CHECK` que exige que exactamente una tenga valor.
4	`nombre_servicio` en la relación *incluye* repetía el nombre del servicio.	Se elimina. En la sección 5 se ve que viola 2FN.
5	La tarifa por persona adicional no se puede aplicar sin saber cuántas personas hay.	Se agrega `reserva.cantidad_personas`.

3. Punto de partida: relación sin normalizar (0FN)

Para ver el proceso completo, imagina que todo se guardara en una sola tabla, como el comprobante impreso de una reserva:

```
COMPROBANTE(codigo_reserva, fecha_creacion, inicio, fin, estado, total,
cantidad_personas,
  correo_cliente, nombre_completo_cliente, telefono_cliente,
  nombre_motel, direccion, ciudad, departamento, NIT, razon_social, banco,
numero_cuenta,
  numero_habitacion, piso, tipo_habitacion, capacidad, caracteristicas,
  modalidad, duracion_horas, precio_tarifa, tipo_dia,
  porcentaje_reembolso, acepta_reembolso, ...política...,
  servicios, pagos, calificacion, comentario, respuesta)
```

codigo	cliente	motel	ciudad	depto	hab	tipo	caracteristicas	modalidad	servicios	pagos
VR-001	Laura Gómez	Luna	Medellín	Antioquia	305	Suite	Jacuzzi, TV, Sauna	Rato (3 h)	Champaña×1 $90.000; Decoración×1 $50.000	Tarjeta $150.000; Efectivo $110.000
VR-002	Andrés Ruiz	Luna	Medellín	Antioquia	210	Estándar	TV	Noche (12 h)	—	Nequi $180.000

En estas dos filas ya aparecen todos los problemas:

- Medellín y Antioquia se repiten en cada reserva del motel.
- Las características, los servicios y los pagos son **listas dentro de una celda**.
- El nombre del cliente está compuesto.
- Si se borra la reserva VR-002, se pierde que existe la habitación 210.

4. Primera forma normal (1FN)

Regla: todos los valores son atómicos, no hay grupos repetidos y cada fila se identifica con una clave.

Pasos:

1. **Atributos compuestos a atómicos:** `nombre_completo` se divide en `nombre` y `apellido`, y `direccion` en `calle` y `carrera`. El ER ya venía así.
2. **Cada grupo repetido sale a su propia tabla, llevándose la clave de su dueño:**
   - `servicios` → `RESERVA_SERVICIO(id_reserva, id_servicio, nombre_servicio, descripcion, precio_lista, cantidad, precio_unitario)`, con PK `(id_reserva, id_servicio)`

- `pagos` → `PAGO(id_pago, id_reserva, monto, fecha_pago, metodo_pago, estado, referencia)`
- `caracteristicas` pertenecen al tipo, no a la reserva → `TIPO_CARACTERISTICA(id_tipo, id_caracteristica, nombre_caract, icono)`, con PK `(id_tipo, id_caracteristica)`
- las fotos (varias URI) → `FOTO(id_foto, id_motel, id_habitacion, uri, orden)`

3. Se usan los identificadores que el ER ya define (`id_usuario`, `id_motel`, etc.).

Resultado 1FN: queda `RESERVA_1FN` con todos los atributos de un solo valor y PK `id_reserva`, más las cuatro tablas anteriores.

5. Segunda forma normal (2FN)

Regla: estar en 1FN y que ningún atributo no primo dependa de solo una *parte* de una clave compuesta.

Solo pueden violar 2FN las tablas con PK compuesta. Las de PK simple (`RESERVA_1FN`, `PAGO`, `FOTO`) ya cumplen 2FN automáticamente.

RESERVA_SERVICIO (id_reserva, id_servicio)

- `id_servicio → nombre_servicio, descripcion, precio_lista` ❌ dependencia parcial
- `(id_reserva, id_servicio) → cantidad, precio_unitario` ✅ dependen de la clave completa

Se descompone en:

- `SERVICIO_ADICIONAL(id_servicio, id_motel, nombre, descripcion, precio, activo)`
- `RESERVA_SERVICIO(id_reserva, id_servicio, cantidad, precio_unitario)`

`nombre_servicio` desaparece (ajuste 4). `precio_unitario` se queda porque es el precio *al momento de la compra*. Si mañana sube `servicio_adicional.precio`, las reservas viejas conservan lo que se cobró. Ese precio depende de la pareja (reserva, servicio), no solo del servicio, así que no es una dependencia parcial.

TIPO_CARACTERISTICA (id_tipo, id_caracteristica)

- `id_caracteristica → nombre, icono` ❌ dependencia parcial

Se descompone en:

- `CARACTERISTICA(id_caracteristica, nombre, icono)`

- `TIPO_HABITACION_CARACTERISTICA(id_tipo, id_caracteristica)`

Anomalía resuelta: antes, cambiar el nombre de un servicio obligaba a editar cada reserva que lo hubiera comprado. Ahora se cambia en una sola fila.

6. Tercera forma normal (3FN)

Regla: estar en 2FN y que, para toda DF X → A, **X sea superclave o A sea un atributo primo**. Es decir, no puede haber dependencias transitivas.

Descomposición de RESERVA_2FN

Estas son las DF transitivas que se encuentran dentro de la tabla grande, y la tabla nueva que sale de cada una:

DF transitiva	Tabla nueva
`id_usuario → correo, nombre, apellido, telefono, rol…`	**usuario**
`id_usuario(propietario) → NIT, razon_social, banco, tipo_cuenta, numero_cuenta…`	**perfil_propietario**
`id_habitacion → numero, piso, estado, id_tipo`	**habitacion**
`id_tipo → nombre, capacidad_personas, id_motel`	**tipo_habitacion**
`id_motel → nombre, calle, carrera, id_ciudad, latitud, longitud…`	**motel**
`id_motel → dias_habiles_devolucion, porcentaje_reembolso…`	**politica_reserva**
`id_tarifa → precio, tipo_dia, vigencia, id_tipo, id_modalidad`	**tarifa**
`id_reserva → calificacion, comentario` (opcional)	**resena**

Hallazgos nuevos de 3FN (no estaban en el diagrama)

a) **`ciudad → departamento`** ❌ dependencia transitiva en MOTEL. Cada motel de Medellín repite "Antioquia", y un error de digitación ("Antiokia") deja la base contradictoria. Se crean `DEPARTAMENTO(id_departamento, nombre)` y `CIUDAD(id_ciudad, id_departamento, nombre)`, y `motel` guarda solo `id_ciudad`. El *nombre* de la ciudad no basta como clave, porque hay municipios homónimos en distintos departamentos (por ejemplo, "La Unión"); por eso la ciudad lleva su propio id.

b) **`porcentaje_reembolso → acepta_reembolso`** ❌ en POLITICA_RESERVA. Si el porcentaje es mayor que 0, acepta reembolso; si es 0, no. Es un atributo derivado: se elimina `acepta_reembolso` y se calcula como `porcentaje_reembolso > 0`.

c) **`nombre_modalidad → duracion_horas`** ❌ en TARIFA. "Rato" siempre dura 3 horas y "Noche" 12. Se crea `MODALIDAD(id_modalidad, nombre, duracion_horas)`. *Supuesto:* la plataforma maneja un catálogo de modalidades. Si cada motel definiera sus propias duraciones, la DF no existiría y el atributo se quedaría en `tarifa`.

Lo que se analizó y no viola 3FN

- **`reserva.fecha_hora_fin`**: podría parecer derivada (inicio + duración), pero la estadía puede extenderse o acortarse, así que no es una DF real. Además, es indispensable para consultar la disponibilidad con solapamiento de fechas.
- **`reserva.total`**: es un valor histórico de lo que se cobró, igual que `precio_unitario`. Si los precios cambian después, el total no debe cambiar.
- **`resena`** no guarda **`id_usuario`** ni **`id_motel`**: se obtienen por la reserva. Guardarlos sería una dependencia transitiva.
- **`respuesta_propietario`**: es débil y 1:1 con la reseña, así que su PK es `id_resena`. El `id_respuesta` sobra.
- **`habitacion.estado`**: solo guarda estados operativos (ACTIVA, MANTENIMIENTO, FUERA_DE_SERVICIO). "Ocupada" se *calcula* a partir de las reservas vigentes; si se guardara, sería un dato derivado.
- **`habitacion(id_motel, id_tipo)`**: tiene la DF `id_tipo → id_motel`, pero **sí cumple 3FN**, porque `id_motel` es primo: forma parte de la clave candidata `{id_motel, numero}`. Se analiza en la sección 8.

7. Esquema final en 3FN (19 tablas)

#	Tabla	PK	FK	UNIQUE / CHECK
1	departamento	id_departamento	—	UQ nombre

#	Tabla	PK	FK	UNIQUE / CHECK
2	ciudad	id_ciudad	id_departamento	UQ (id_departamento, nombre)
3	usuario	id_usuario	—	UQ correo, UQ telefono (ya implementada)
4	perfil_propietario	**id_usuario**	id_usuario → usuario	UQ nit, UQ (banco, numero_cuenta)
5	motel	id_motel	id_propietario → perfil_propietario, id_ciudad	—
6	politica_reserva	**id_motel**	id_motel → motel	CHECK porcentaje 0–100
7	tipo_habitacion	id_tipo	id_motel	UQ (id_motel, nombre), UQ (id_tipo, id_motel)
8	caracteristica	id_caracteristica	—	UQ nombre
9	tipo_habitacion_caracteristica	(id_tipo, id_caracteristica)	ambas	—
10	habitacion	id_habitacion	**(id_tipo, id_motel) → tipo_habitacion(id_tipo, id_motel)**	UQ (id_motel, numero)
11	foto	id_foto	id_motel NULL, id_habitacion NULL	CHECK exactamente una no nula; UQ (id_motel, orden), UQ (id_habitacion, orden)
12	modalidad	id_modalidad	—	UQ nombre
13	tarifa	id_tarifa	id_tipo, id_modalidad	CHECK vigente_hasta ≥ vigente_desde
14	servicio_adicional	id_servicio	id_motel	UQ (id_motel, nombre)
15	reserva	id_reserva	id_usuario, id_habitacion, id_tarifa	UQ codigo_reserva; CHECK fin > inicio
16	reserva_servicio	(id_reserva, id_servicio)	ambas	CHECK cantidad > 0
17	pago	id_pago	id_reserva	UQ referencia_transaccion
18	resena	id_resena	id_reserva	**UQ id_reserva** (1:1); CHECK calificación 1–5
19	respuesta_propietario	**id_resena**	id_resena → resena	—

Dos detalles de MySQL que conviene conocer:

- En `foto`, el `CHECK` sobre columnas FK impide usar `ON DELETE CASCADE`, porque MySQL prohíbe acciones referenciales en columnas que usa un CHECK. Las fotos se borran desde el service antes de borrar la habitación.
- Que la tarifa de una reserva corresponda al mismo tipo que su habitación no se puede declarar sin repetir `id_tipo` en `reserva`, y repetirlo violaría 3FN. Esa regla se valida en el service.

8. Evaluación de Boyce-Codd (BCNF)

Llevar el modelo a BCNF no es rentable: 18 de las 19 tablas ya la cumplen, y la única que no (`habitacion`) perdería una regla de integridad si se descompone.

Regla: para toda DF no trivial X → A, X debe ser superclave. La diferencia con 3FN es que BCNF ya no admite la excepción de "A es primo".

Las 18 tablas cumplen porque todos sus determinantes son claves candidatas. Esto incluye las que tienen varias claves:

- `usuario`: id_usuario, correo y telefono
- `perfil_propietario`: id_usuario, nit, y (banco, numero_cuenta)
- `reserva`: id_reserva y codigo_reserva
- `resena`: id_resena e id_reserva
- `tipo_habitacion`: id_tipo y (id_motel, nombre)

La excepción: habitacion

- Claves candidatas: `{id_habitacion}` y `{id_motel, numero}`.
- DF `id_tipo → id_motel`, e `id_tipo` no es superclave ❌ BCNF. Sí cumple 3FN, porque `id_motel` es primo.

Para llevarla a BCNF habría que quitar `id_motel` de `habitacion`. Eso cuesta:

- **Perder la regla `UNIQUE(id_motel, numero)`:** se podrían crear dos "habitación 305" en el mismo motel. La descomposición **no preserva dependencias**, y la regla tendría que vigilarla un trigger.
- **Un JOIN extra** con `tipo_habitacion` en cada consulta de "habitaciones del motel X".

Decisión: se queda en 3FN con la **FK compuesta `(id_tipo, id_motel)`**. Así la redundancia queda controlada por el motor: `id_motel` nunca puede contradecir el motel del tipo. Es el caso clásico en el que 3FN es preferible a BCNF.

9. Evaluación de 4FN y 5FN

El modelo ya cumple 4FN y 5FN, así que llegar a ellas no cuesta nada.

4FN: dependencias multivaluadas (X ↠ Y)

Aparecen cuando una tabla guarda dos listas independientes de una misma entidad. Si existiera `TIPO(id_tipo, caracteristica, tarifa)`, un tipo con 3 características y 4 tarifas necesitaría 12 filas: el producto cartesiano.

En este modelo eso nunca ocurre, porque cada lista quedó en su propia tabla desde 1FN:

- `tipo_habitacion_caracteristica` es binaria y toda su clave son sus columnas, así que solo tiene dependencias multivaluadas triviales.
- En `reserva_servicio`, `cantidad` y `precio_unitario` dependen de la pareja, así que no son independientes.

5FN: dependencias de reunión

Una tabla viola 5FN cuando solo se puede reconstruir uniendo **tres o más** proyecciones. Pasa con relaciones **ternarias** que tienen reglas cíclicas, como el ejemplo clásico proveedor–pieza–proyecto. Este ER solo tiene relaciones **binarias**, y cada tabla describe una entidad o una relación binaria.

Además, el teorema de Date y Fagin dice que una tabla en 3FN cuyas claves candidatas son todas simples está automáticamente en 5FN. Eso cubre la mayoría de las tablas. En las demás, las claves alternativas compuestas (los UNIQUE) dependen funcionalmente de la PK, así que no hay ninguna dependencia de reunión que no esté implicada por las claves.

10. Conclusión y decisiones para la sustentación

El objetivo del proyecto es 3FN, y el modelo la cumple en las 19 tablas.

Forma normal	¿Se cumple?	Costo de llegar	Decisión
1FN	Sí	Separar listas y atributos compuestos	Obligatoria
2FN	Sí	2 descomposiciones (servicios, características)	Obligatoria
3FN	Sí	Ciudad/departamento, modalidad, quitar `acepta_reembolso` y `nombre_servicio`	**Objetivo del proyecto**
BCNF	18 de 19 tablas	Perder `UNIQUE(id_motel, numero)` en `habitacion`	No rentable: 3FN con FK compuesta
4FN	Sí	Ninguno	Ya se cumple
5FN	Sí	Ninguno	Ya se cumple

Tres decisiones de diseño que conviene poder defender si el profesor pregunta:

1. `precio_unitario` y `total` son **datos históricos**, no redundancia.
2. `fecha_hora_fin` **no es derivada**, porque la estadía puede cambiar.

3. `habitacion` se queda en 3FN **a propósito**, con la FK compuesta que impide inconsistencias.